

MD005 - Sistemes Basats en el Coneixement

Tercera Entrega: Models de Regressió



Alumne: Gerard Pascual Fontanilles

Professora: Jordina Torrents

Data d'entrega: 30 de gener de 2026

Informació sobre el conjunt de dades

Per fer aquest treball he tornat a utilitzar el conjunt de dades de pokemons. Extret de:
<https://www.kaggle.com/datasets/hamzacyberpatcher/data-of-1010-pokemons>

Tornem a repassar el conjunt de dades: conté informació sobre 1025 pokemons diferents, incloent-hi les seves estadístiques bàsiques, tipus, generació i si evolucionen d'algun altre pokemon.

Les característiques principals del conjunt de dades són:

- Pokemon's id
- Pokemon's name
- Pokemon's rank (whether it is legendary, mythical, baby or an ordinary pokemon)
- Pokemon's generation
- The pokemon from which this pokemon evolved from
- It's primary and secondary type as type1 and type2
- It's base stats
- It's height (decimetres) and weight (hectograms)
- It's abilities (Note: if a pokemon has multiple abilities such as overgrow and chlorophyll than they are separated by a space and if there is a space in the name of the ability it has been replaced by a hyphen for example a pokemon with abilities blaze and solar power it is given as blaze solar-power)
- A little description of the pokemon in english (description of pokemons from 1009 is marked as "Not Available").

Imports necessaris

```
In [1]: # --- 1. LLIBRERIES ESTÀNDARD I MANIPULACIÓ DE DADES ---
import pandas as pd
import numpy as np
import scipy.stats as stats
from scipy.stats import randint
import warnings

# --- 2. VISUALITZACIÓ GRÀFICA ---
```

```

import matplotlib.pyplot as plt
import seaborn as sns

# --- 3. SCIKIT-LEARN: PREPROCESSAMENT I PIPELINES ---
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

# --- 4. SCIKIT-LEARN: MODELS (Regressió i Classificació) ---
from sklearn.linear_model import Ridge
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier

# --- 5. SCIKIT-LEARN: SELECCIÓ DE MODELS I OPTIMITZACIÓ ---
from sklearn.model_selection import train_test_split
from sklearn.experimental import enable_halving_search_cv # Necessari per activar Ha
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV, HalvingRandomSe

# --- 6. SCIKIT-LEARN: MÈTRIQUES I EXCEPCIONS ---
from sklearn.metrics import (
    mean_squared_error, mean_absolute_error, r2_score, # Mètriques Regressió
    accuracy_score, confusion_matrix, classification_report # Mètriques Classificació
)
from sklearn.exceptions import UndefinedMetricWarning

```

Introducció: Canvi d'Enfocament

Encara que utilitzem el mateix conjunt de dades que en l'entrega anterior, l'objectiu d'aquesta pràctica és conceptualment diferent.

- **En la Classificació:** L'objectiu era la **discriminació**. Buscàvem separar els Pokémon en categories discretes (*Llegendari vs. Ordinari*). Ens fixàvem en la separabilitat de les classes i en traçar fronteres de decisió.
- **En la Regressió (Aquesta):** L'objectiu és la **predicció numèrica**. Ara busquem estimar un valor continu (per exemple, predir l'estadística de *Velocitat* o l'*Atac* basant-nos en la resta de característiques).

Per tant, l'anàlisi de dades canvia:

1. **Correlació vs. Separació:** Ja no busquem variables que separin grups, sinó variables que tinguin una **correlació lineal** o directa amb el nostre objectiu numèric (ex: si augmenta el pes, disminueix la velocitat?).
2. **Sensibilitat al Soroll:** En classificació, un outlier extrem podia ser fàcil de classificar. En regressió, un valor extrem pot desviar completament la línia de predicció i disparar l'error quadràtic mitjà (MSE), per la qual cosa la gestió d'outliers serà més crítica.

Neteja i Preparació de Dades

Partim del coneixement adquirit en l'anàlisi anterior, però adaptem la neteja per maximitzar la capacitat predictiva numèrica:

1. **Dades Nul·les (type2):** Sabem que la columna **type2** conté molts valors nuls. En lloc d'eliminar aquestes files, imputarem el valor **'None'**. Això és important perquè, en regressió, el fet de ser un Pokémon "pur" (un sol tipus) pot tenir una correlació amb la distribució de les seves estadístiques.
2. **Transformació d'Evolucions (evolves_from -> is_base):**

- La columna `evolves_from` original conté text (noms de Pokémon), la qual cosa genera un soroll excessiu (massa categories úniques).
- No obstant això, **l'estat evolutiu és el predictor més fort de les estadístiques** (un Pokémon evolucionat és quasi sempre més fort que la seva base).
- Crearem, una altra vegada, una nova variable numèrica `is_base` (1 si no evoluciona de ningú, 0 si és una evolució) i eliminarem la columna de text original. Així conservem el senyal matemàtic sense el soroll del text.

3. Eliminació de Soroll:

- Eliminarem identificadors com `id`, `name`, `desc` i `abilities`. Aquestes variables no tenen una relació lineal directa amb les estadístiques numèriques i complicarien innecessàriament el model.

```
In [2]: df = pd.read_csv('pokemons.csv')
```

```
In [3]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                    1025 non-null   int64
1   name                  1025 non-null   object
2   rank                  1025 non-null   object
3   generation            1025 non-null   object
4   evolves_from          1025 non-null   object
5   type1                 1025 non-null   object
6   type2                 526 non-null    object
7   hp                    1025 non-null   int64
8   atk                   1025 non-null   int64
9   def                   1025 non-null   int64
10  spatk                 1025 non-null   int64
11  spdef                 1025 non-null   int64
12  speed                 1025 non-null   int64
13  total                 1025 non-null   int64
14  height                1025 non-null   int64
15  weight                1025 non-null   int64
16  abilities              1025 non-null   object
17  desc                  1025 non-null   object
dtypes: int64(10), object(8)
memory usage: 144.3+ KB
```

```
In [4]: nuls = df.isnull().sum()
print(nuls[nuls > 0])
```

```
type2    499
dtype: int64
```

```
In [5]: if 'type2' in df.columns:
        df['type2'] = df['type2'].fillna('None')
```

```
In [6]: df['is_base'] = df['evolves_from'].apply(lambda x: 1 if x == 'nothing' else 0)
print(f"{df['is_base'].value_counts()}")
```

```
is_base
1     541
0     484
Name: count, dtype: int64
```

```
In [7]: cols_to_drop = ['id', 'name', 'desc', 'abilities', 'evolves_from']
```

```
existing_drop = [c for c in cols_to_drop if c in df.columns]
df.drop(columns=existing_drop, inplace=True)
print(f"Columnes eliminades: {existing_drop}")
```

Columnes eliminades: ['id', 'name', 'desc', 'abilities', 'evolves_from']

```
In [8]: duplicats = df.duplicated().sum()
if duplicats > 0:
    df.drop_duplicates(inplace=True)
    print(f"S'han eliminat {duplicats} files duplicades.")
else:
    print("No hi ha files duplicades.")
```

No hi ha files duplicades.

```
In [9]: df.head()
```

```
Out[9]:
```

	rank	generation	type1	type2	hp	atk	def	spatk	spdef	speed	total	height	weight	is_
0	ordinary	generation-	grass	poison	45	49	49	65	65	45	318	7	69	
1	ordinary	generation-	grass	poison	60	62	63	80	80	60	405	10	130	
2	ordinary	generation-	grass	poison	80	82	83	100	100	80	525	20	1000	
3	ordinary	generation-	fire	None	39	52	43	60	50	65	309	6	85	
4	ordinary	generation-	fire	None	58	64	58	80	65	80	405	11	190	

```
In [10]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
#   Column          Non-Null Count  Dtype
---  -
0   rank             1025 non-null   object
1   generation       1025 non-null   object
2   type1            1025 non-null   object
3   type2            1025 non-null   object
4   hp               1025 non-null   int64
5   atk              1025 non-null   int64
6   def              1025 non-null   int64
7   spatk            1025 non-null   int64
8   spdef            1025 non-null   int64
9   speed            1025 non-null   int64
10  total            1025 non-null   int64
11  height           1025 non-null   int64
12  weight           1025 non-null   int64
13  is_base          1025 non-null   int64
dtypes: int64(10), object(4)
memory usage: 112.2+ KB
```

Anàlisi Exploratòria de Dades (EDA): Enfocament per a Regressió

Tot i que treballem amb el mateix dataset que en l'entrega anterior, l'objectiu matemàtic ha canviat radicalment, la qual cosa ens obliga a interrogar les dades d'una manera diferent (tornem a posar un exemple més clar):

- **En Classificació (Entrega Anterior):** Buscàvem **Discriminació i Separabilitat**.

- *Pregunta*: "És l'Atac dels Llegendaris significativament diferent del dels Ordinaris?"
- *Eina*: Boxplots i gràfics de barres per veure si les mitjanes estaven allunyades.
- **En Regressió (Aquesta Entrega)**: Busquem **Correlació i Linealitat**.
 - *Pregunta*: "Quan augmenta el Pes d'un Pokémon, quant augmenta exactament el seu Atac?"
 - *Eina*: Scatterplots i Correlació de Pearson. Ja no ens importa separar grups, sinó modelar la tendència contínua de les dades.

Per validar si podem aplicar un model de Regressió Lineal amb garanties, executarem l'anàlisi en tres fases seqüencials:

1. Visió Global (Mapa de Calor):

- Calcularem la matriu de correlació per identificar quines variables tenen una relació forta amb el nostre objectiu (*Target*) i, molt important, vigilar la **Multicolinealitat**.
 - Si dues variables predictores (ex: **Defense** i **Sp. Def**) tenen una correlació excessiva (> 0.85), aporten informació redundant que pot desestabilitzar els coeficients del model.

2. Comprovació de Linealitat (Pairplots):

- Generarem gràfics de dispersió (*scatter plots*) creuats.
- Busquem "núvols" de punts que segueixin una tendència recta (ascendent o descendent). Si veiem formes de "U" o núvols totalment rodons, sabrem que la regressió lineal tindrà dificultats.

3. Validació de Normalitat (Histogrames + QQ-Plots):

- Aquesta és la prova més rigorosa. Els models de regressió assumeixen que les variables (o els seus errors) segueixen una **Distribució Normal** (Campana de Gauss).
- Utilitzarem **QQ-Plots** al costat dels Histogrames. Si els punts del QQ-Plot es desvien de la línia diagonal, sabrem que tenim problemes d'asimetria (*Skewness*) o *outliers*, i hauré d'aplicar transformacions matemàtiques (com el logaritme) per corregir-ho.

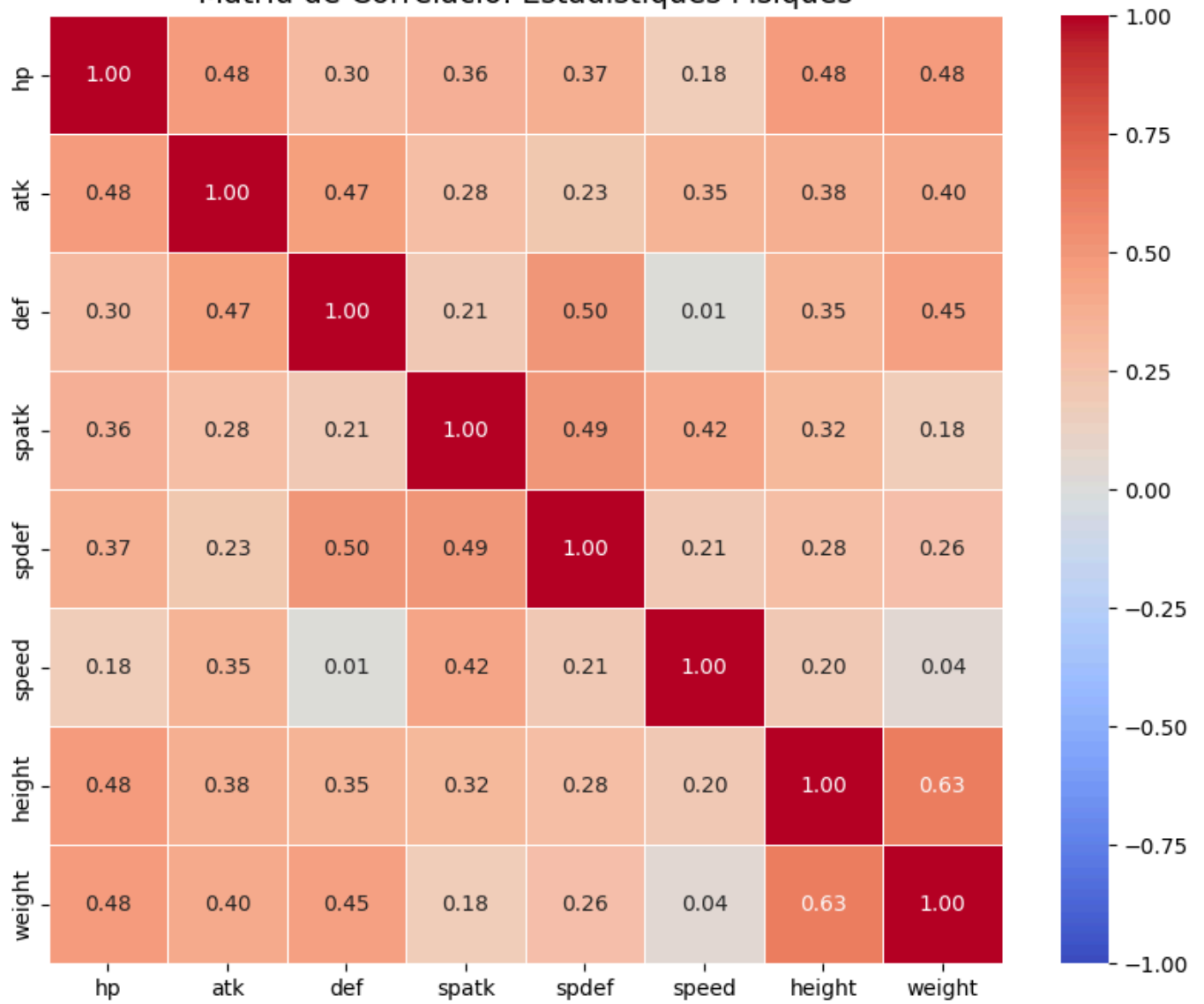
```
In [11]: numeric_cols = ['hp', 'atk', 'def', 'spatk', 'spdef', 'speed', 'height', 'weight']
df_num = df[numeric_cols]
```

```
In [12]: plt.figure(figsize=(10, 8))
correlation_matrix = df_num.corr()

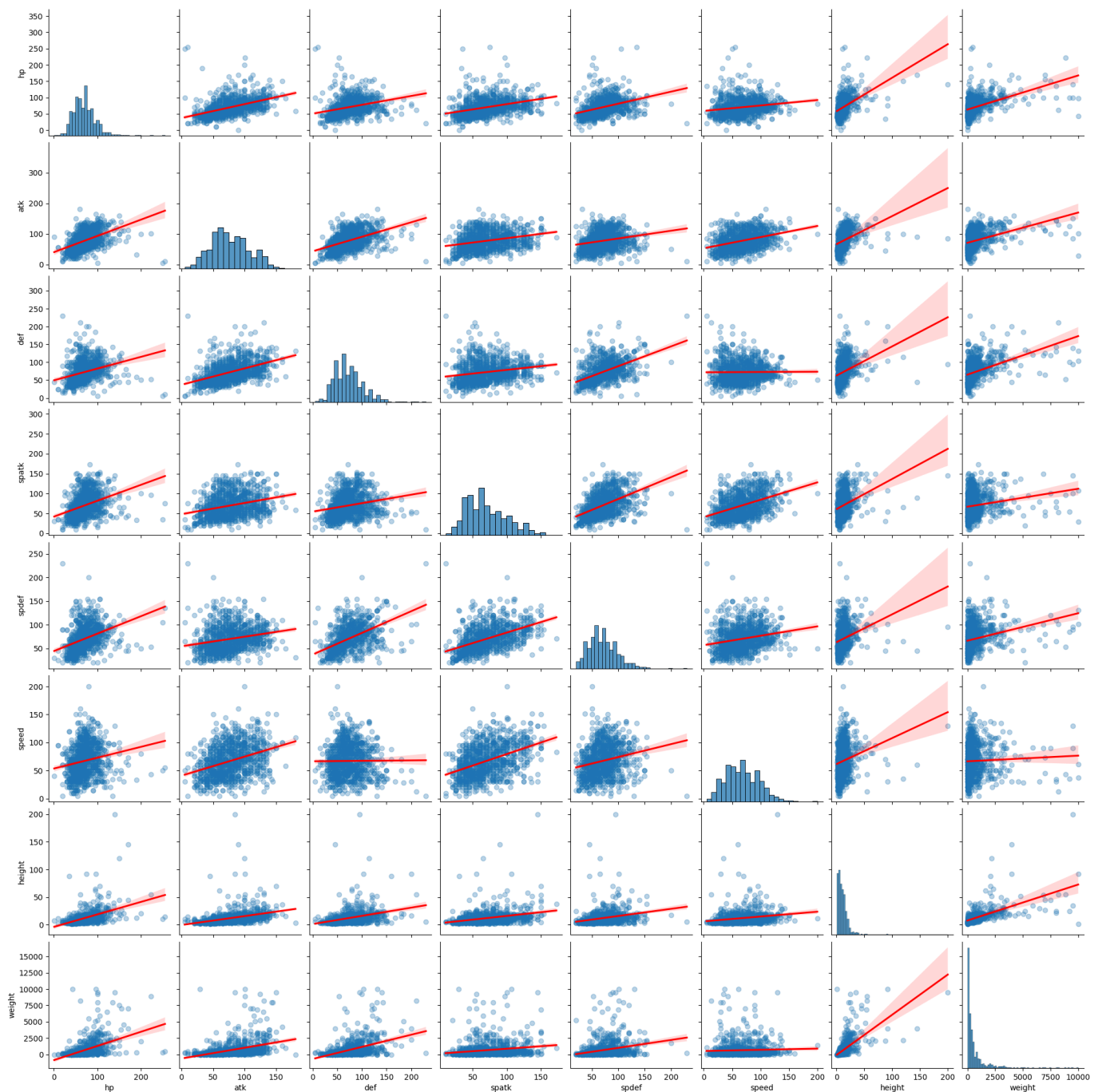
sns.heatmap(correlation_matrix,
            annot=True,
            fmt=".2f",
            cmap='coolwarm',
            vmin=-1, vmax=1,
            linewidths=0.5)

plt.title('Matriu de Correlació: Estadístiques Físiques', fontsize=14)
plt.show()
```

Matriu de Correlació: Estadístiques Físiques



```
In [13]: sns.pairplot(df_num,
                    kind='reg',
                    plot_kws={'line_kws':{'color':'red'}, 'scatter_kws': {'alpha': 0.3}})
plt.suptitle("Relacions Lineals entre Variables", y=1.02, fontsize=16)
plt.show()
```

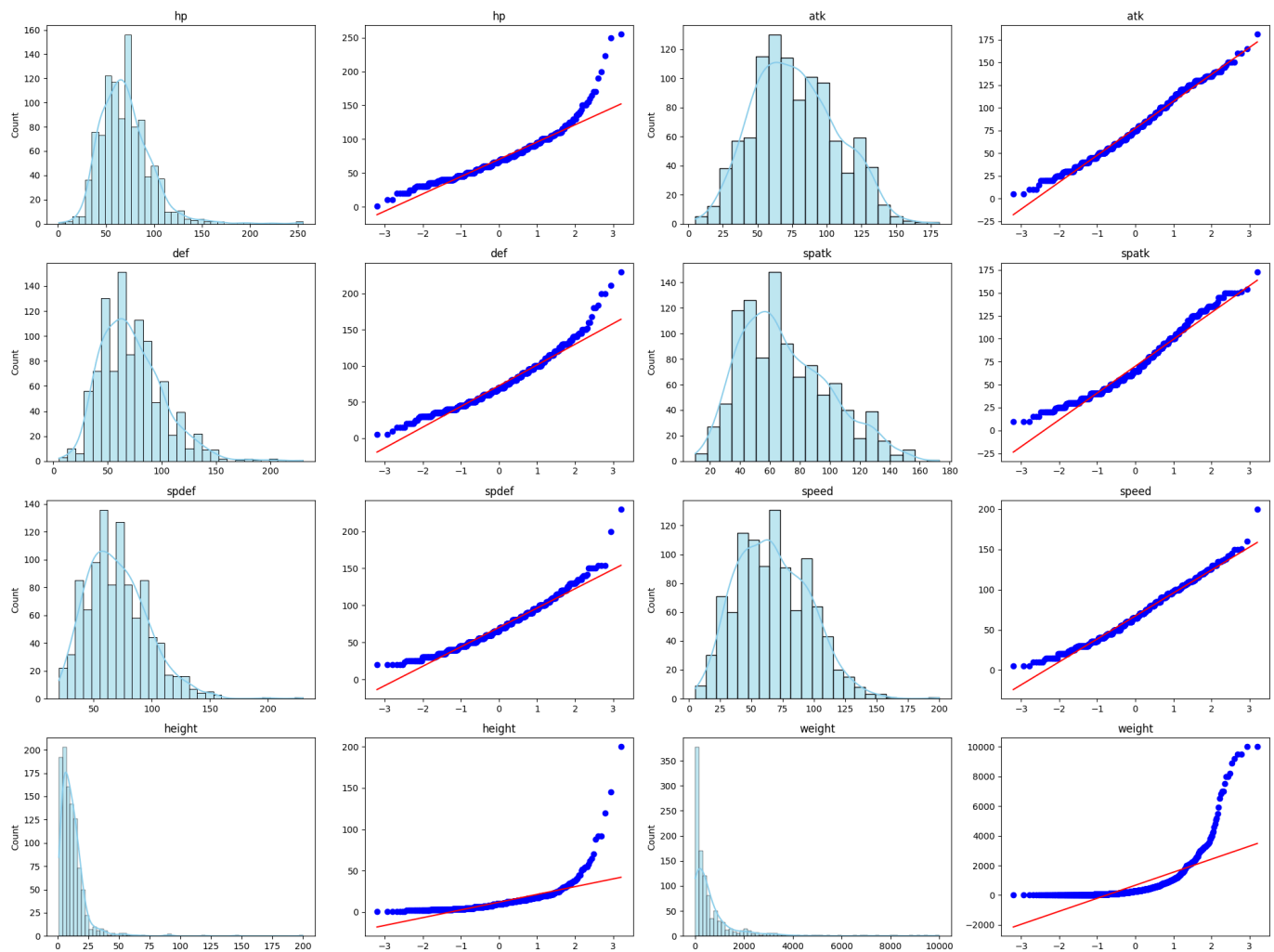


```
In [14]: plt.figure(figsize=(20, 15))

for i, col in enumerate(numeric_cols):
    plt.subplot(4, 4, 2*i + 1)
    sns.histplot(df[col], kde=True, color='skyblue')
    plt.title(f'{col}')
    plt.xlabel('')

    plt.subplot(4, 4, 2*i + 2)
    stats.probplot(df[col], dist="norm", plot=plt)
    plt.title(f'{col}')
    plt.xlabel('')
    plt.ylabel('')

plt.tight_layout()
plt.suptitle("Anàlisi de Normalitat: Histograma vs QQ-Plot", y=1.02, fontsize=16)
plt.show()
```

L'anàlisi conjunta visual (**Pairplots**) i estadística (**QQ-Plots**) ens permet extreure conclusions robustes sobre la naturalesa de les dades:

1. Correlacions (Relacions Lineals):

- La correlació més alta trobada és de **0.63** (entre **height** i **weight**). Com que està lluny del llindar perillós de 0.85-0.90, **no cal eliminar cap variable per redundància**. Totes aporten informació prou única al model.
- La variable objectiu (**atk**) mostra correlacions positives moderades amb **hp** (0.48), **def** (0.47) i **weight** (0.40). Això confirma la hipòtesi que la constitució física és un bon predictor: els Pokémon més grossos i resistents tendeixen a tenir més força física.

2. Anàlisi Visual (Pairplots):

- Per a les variables normals (**atk**, **def**...), els núvols de punts tenen forma ovalada amb tendència ascendent, validant l'ús de models lineals.
- En els gràfics que involucren **weight** o **height**, observem que el núvol de punts no ocupa tot l'espai, sinó que queda **"aixafat" contra els eixos**. Això passa perquè el 95% de les dades estan comprimides en un rang molt petit (0-200kg), mentre que l'espai visual s'ha d'estirar fins a 10.000kg només per encabir els outliers. Això dificulta que el model capti la tendència real de la majoria.

3. Validació de Normalitat (QQ-Plots vs Histogrames):

- Els QQ-Plots de variables com **atk**, **speed**, **def** i **spdef** mostren que els punts blaus s'ajusten gairebé perfectament a la línia vermella de 45 graus. Això confirma matemàticament el que intuïem als histogrames: segueixen una **Distribució Normal (Gaussiana)**, l'escenari ideal per a la regressió lineal.

- **L'Excepció Crítica (height i weight)**: En aquests dos casos, el QQ-Plot mostra una **curvatura extrema**. Els punts es desvien dràsticament de la recta, confirmant que NO són normals.
- Això es deu a un biaix positiu extrem (*Right Skew*). La gran majoria de Pokémon són petits/lleugers (concentració a l'esquerra), però existeix un petit grup de "Kaijus" (Pokémon gegants com *Wailord* o *Eternatus*) que distorsionen l'escala, actuant com a *outliers* massius.

Justificació de la Transformació Logarítmica

Donats els resultats anteriors, hem decidit aplicar una transformació matemàtica (`np.log1p`) exclusivament a les variables **Pes (weight)** i **Alçada (height)**.

Per què fem això?

1. Correcció de l'Asimetria (Skewness):

- La regressió lineal és molt sensible als *outliers*. Un Pokémon que pesa 10.000kg (quan la mitjana és 60kg) actuaria com una "palanca", desplaçant la recta de regressió per intentar ajustar-se a ell sol, empitjorant la predicció per als altres 1000 Pokémon.
- El **logaritme** comprimeix les escales grans. Converteix distàncies multiplicatives en distàncies additives. Això "apropa" els valors gegants cap a la mitjana, convertint la distribució de cua llarga en una **Campana de Gauss (Normal)**.

2. Millora de la Linealitat:

- Moltes relacions a la natura segueixen lleis de potència (el pes creix al cub de l'alçada). En aplicar logaritmes, linealitzem aquestes relacions exponencials.
- Això permet que un model lineal simple pugui capturar patrons complexos que, en l'escala original, semblaven corbes o núvols dispersos.

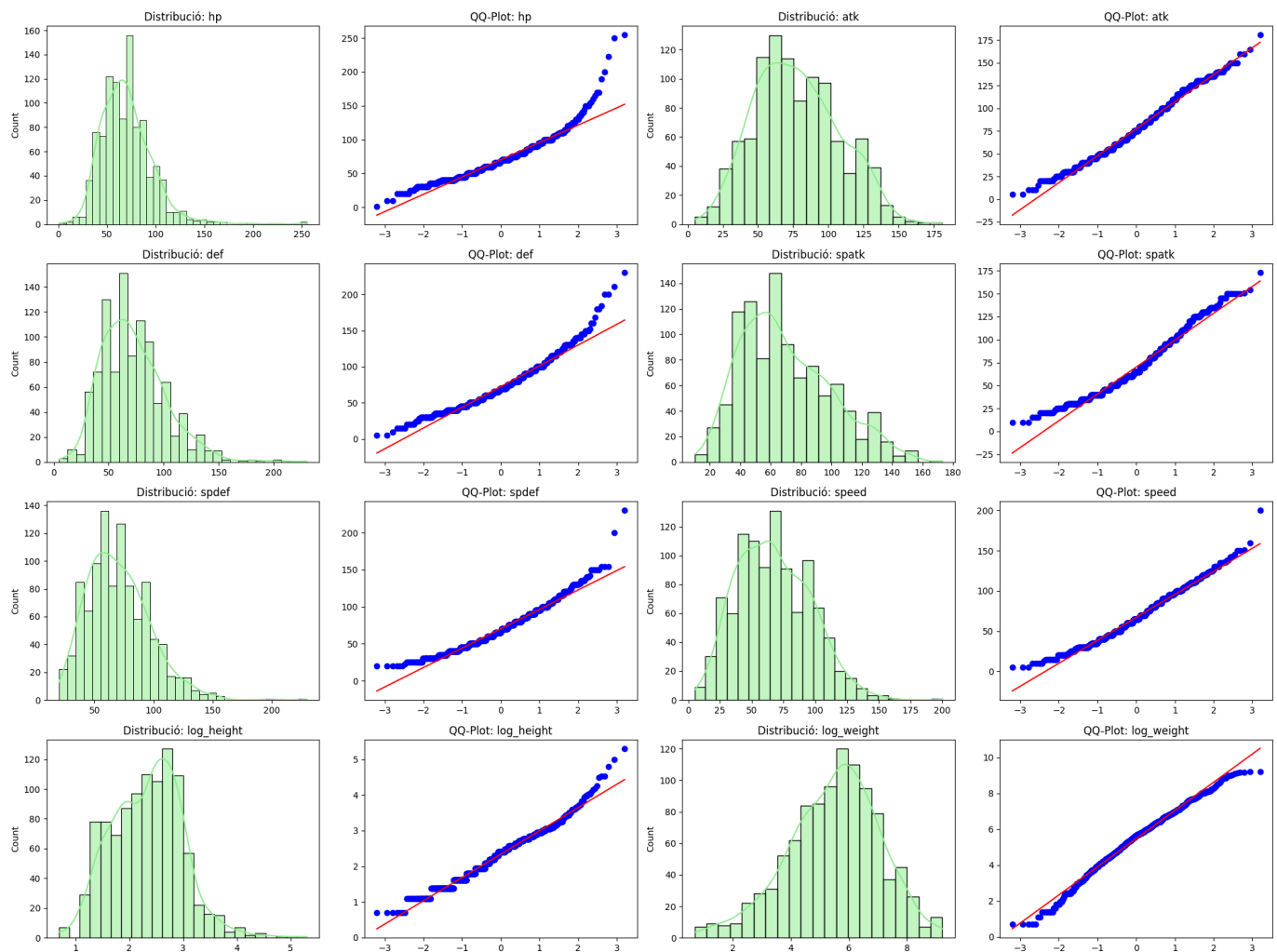
```
In [15]: df['log_height'] = np.log1p(df['height'])
df['log_weight'] = np.log1p(df['weight'])
```

```
In [16]: final_numeric_cols = ['hp', 'atk', 'def', 'spatk', 'spdef', 'speed', 'log_height', 'l
df_num_final = df[final_numeric_cols]
plt.figure(figsize=(20, 15))

for i, col in enumerate(final_numeric_cols):
    plt.subplot(4, 4, 2*i + 1)
    sns.histplot(df[col], kde=True, color='lightgreen')
    plt.title(f'Distribució: {col}')
    plt.xlabel('')

    plt.subplot(4, 4, 2*i + 2)
    stats.probplot(df[col], dist="norm", plot=plt)
    plt.title(f'QQ-Plot: {col}')
    plt.xlabel('')
    plt.ylabel('')

plt.tight_layout()
plt.suptitle("Estat Final de les Variables (Post-Transformació)", y=1.02, fontsize=16)
plt.show()
```



Els gràfics finals confirmen que l'estratègia de transformació ha estat un èxit. La comparativa visual és concloent:

1. Correcció de la Distribució (`log_height` i `log_weight`):

- Si observem els nous **QQ-Plots**, veiem que els punts blaus s'alineen ara gairebé perfectament amb la línia vermella de referència.
- La curvatura extrema que teníem originalment ha desaparegut. Això indica que hem eliminat el biaix positiu excessiu i hem aconseguit una distribució molt propera a la **Normalitat**.

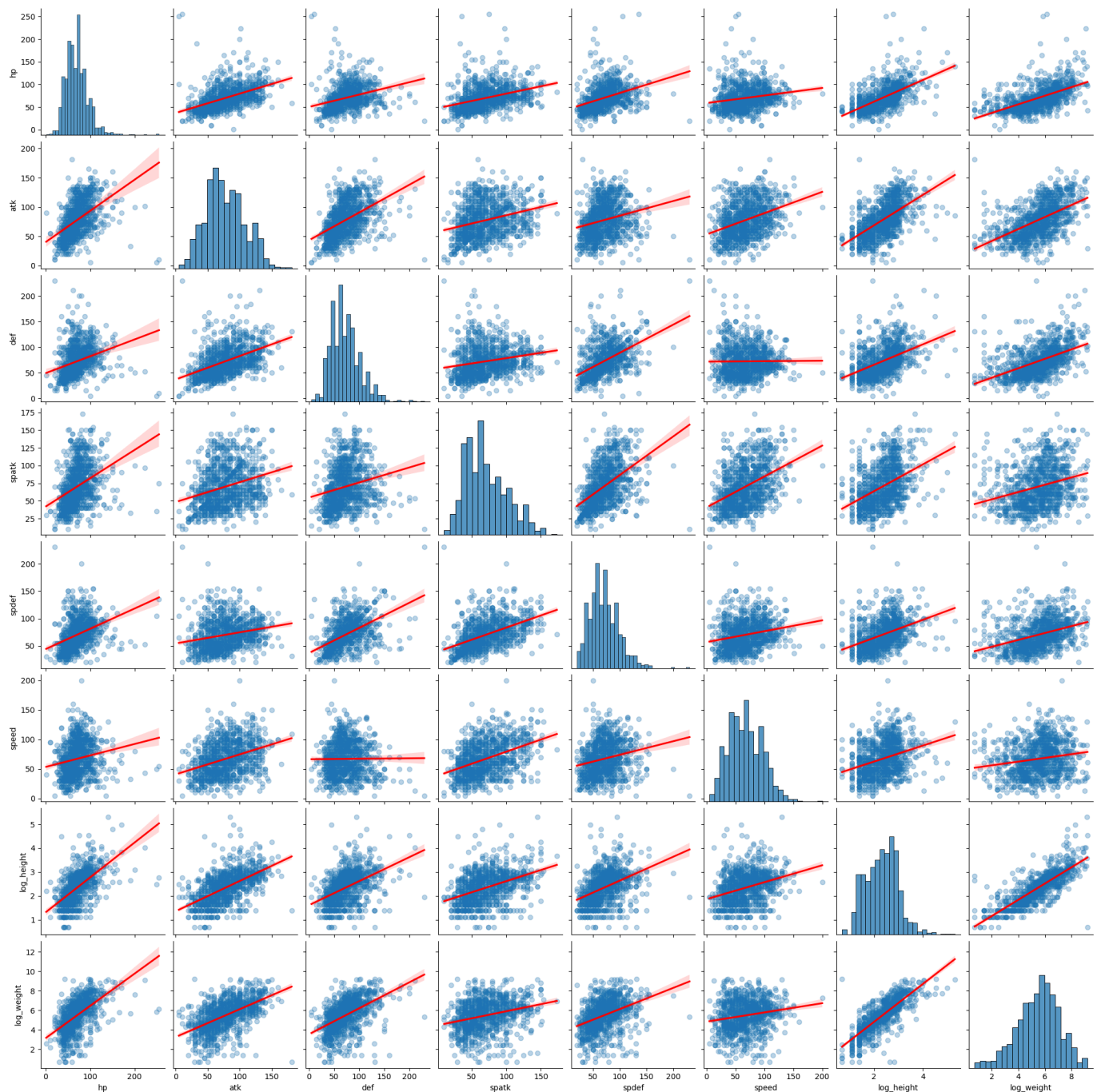
2. Histogrames Centrats:

- Els histogrames ja no mostren una única barra aixafada a l'esquerra. Ara tenen una forma de campana (Gaussiana) molt més definida, amb les dades centrades i les cues simètriques.

Amb aquestes noves variables transformades, hem satisfet els requisits estadístics per a la **Regressió Lineal**. Hem neutralitzat l'efecte nociu dels *outliers* gegants, permetent que el model pugui aprendre patrons generals de manera robusta sense ser distorsionat pels valors extrems.

Un cop aplicades les transformacions logarítmiques a l'Alçada i al Pes, tornem a generar els gràfics de dispersió (**Pairplots**) per verificar l'impacte en la linealitat.

```
In [17]: sns.pairplot(df_num_final,
                    kind='reg',
                    plot_kws={'line_kws':{'color':'red'}, 'scatter_kws':{'alpha': 0.3}})
plt.suptitle("Relacions Lineals entre Variables", y=1.02, fontsize=16)
plt.show()
```



- Si observem les files de `log_height` i `log_weight`, veiem que els punts ja no estan comprimits contra els eixos. Ara formen "núvols" de punts ovalats i centrats.
- **Millora de la Linealitat amb l'Objectiu:** La relació entre `log_weight` i l'Atac (`atk`) és ara molt més clara i rectilínia. La línia de regressió vermella travessa el centre de la distribució, indicant que la correlació és constant en tot el rang de dades.

- Les variables transformades són predictors molt més fiables per a un model lineal que les originals.

2.4. Anàlisi de Variables Categòriques

Abans de procedir a la selecció final de variables, és imprescindible analitzar l'impacte de les característiques qualitatives (`type1`, `generation`, `is_base`) sobre la nostra variable objectiu (**Atac**).

A diferència de les variables numèriques (on miràvem correlacions), aquí busquem **diferències en la distribució**:

- *Influeix l'Element?* Esperem veure que certs tipus (com Drac o Lluita) tinguin una mitjana d'atac superior als altres.
- *Influeix l'Evolució?* La variable `is_base` hauria de mostrar una separació dràstica: els Pokémon evolucionats (0) haurien de tenir estadístiques molt superiors als base (1).

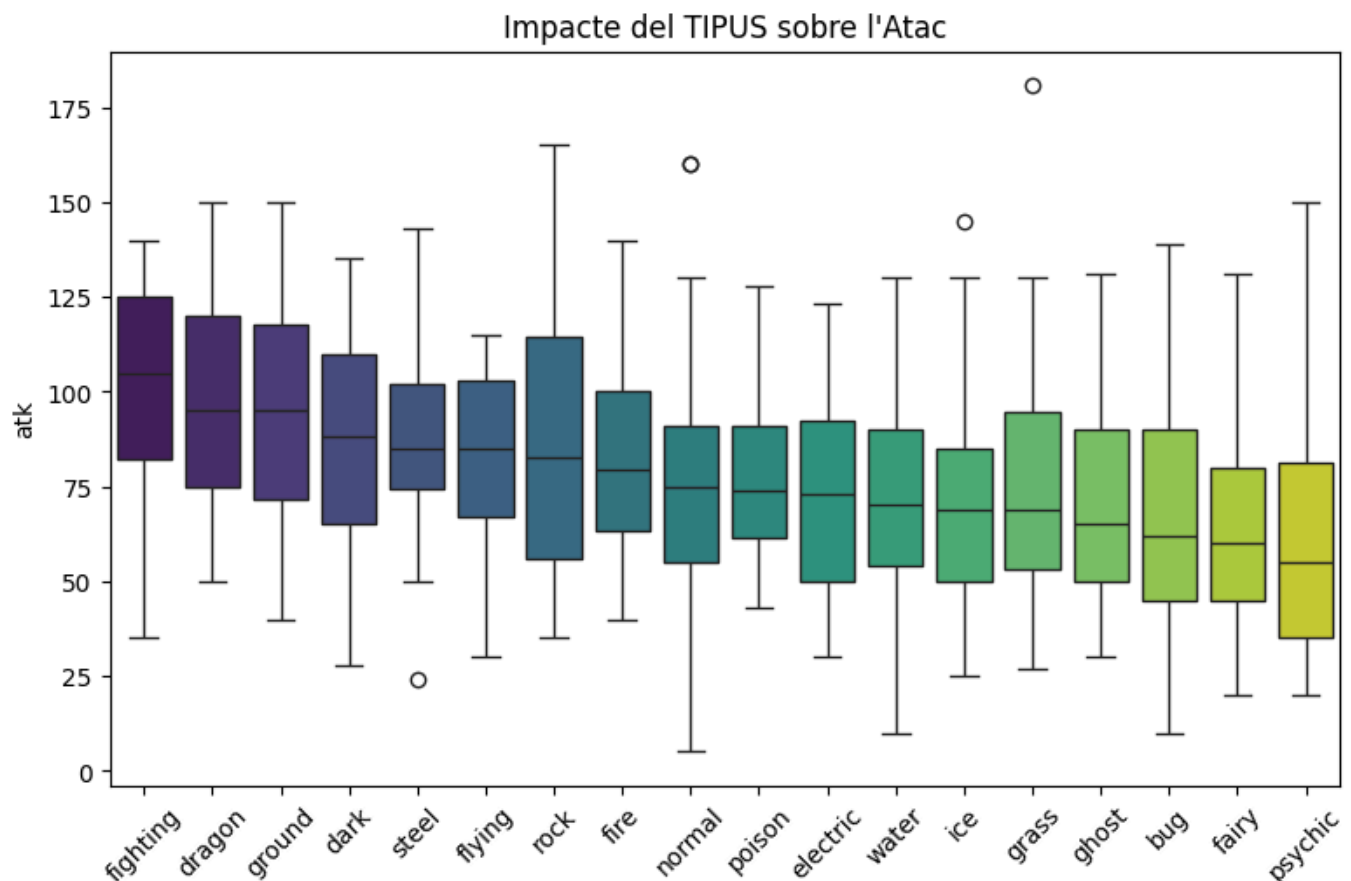
Utilitzarem **Boxplots (Diagrames de Caixa)** ordenats per la mitjana d'atac per visualitzar aquestes jerarquies.

```
In [18]: target = 'atk'

plt.figure(figsize=(20, 12))

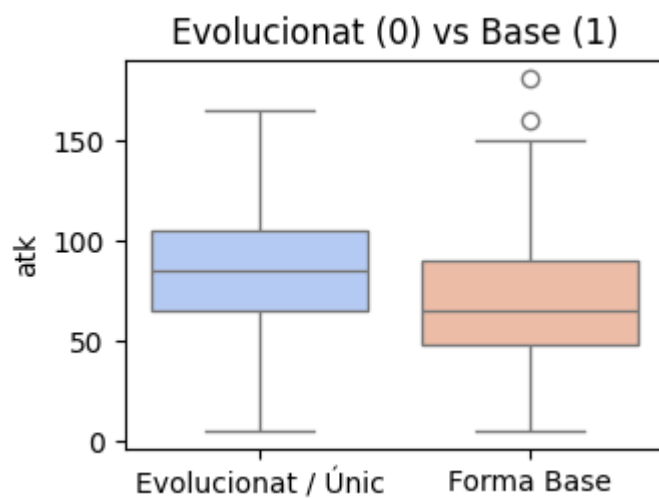
order_type = df.groupby('type1')[target].median().sort_values(ascending=False).index

plt.subplot(2, 2, 1)
sns.boxplot(data=df, x='type1', y=target, order=order_type, palette='viridis')
plt.title(f'Impacte del TIPUS sobre l\'Atac', fontsize=12)
plt.xticks(rotation=45)
plt.xlabel('')
```



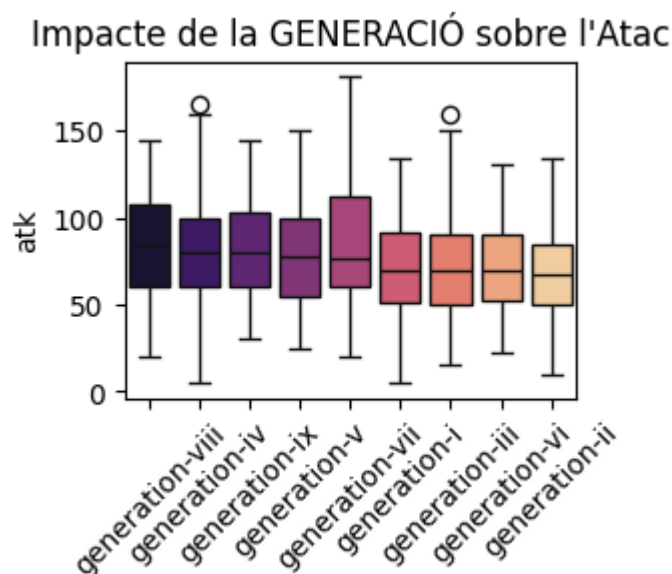
```
In [19]: plt.subplot(2, 2, 3)
sns.boxplot(data=df, x='is_base', y=target, palette='coolwarm')
plt.title(f'Evolucionat (0) vs Base (1)', fontsize=12)
plt.xticks([0, 1], ['Evolucionat / Únic', 'Forma Base'])
plt.xlabel('')

plt.tight_layout()
plt.show()
```



```
In [20]: order_gen = df.groupby('generation')[target].median().sort_values(ascending=False).in

plt.subplot(2, 2, 2)
sns.boxplot(data=df, x='generation', y=target, order=order_gen, palette='magma')
plt.title(f'Impacte de la GENERACIÓ sobre l'Atac', fontsize=12)
plt.xticks(rotation=45)
plt.xlabel('')
```



Conclusions de l'Anàlisi Categòrica

L'anàlisi visual confirma que les variables categòriques aporten informació predictiva crucial i complementària a les dades físiques:

1. Jerarquia de Tipus (`type1`):

- S'observa una discriminació clara entre elements. Tipus associats al combat físic com **Fighting** (Lluita), **Dragon** (Drac) i **Ground** (Terra) dominen la part alta del gràfic amb mitjanes superiors als 90-100 punts.
- Per contra, tipus més "màgics" o petits com **Fairy** (Fada), **Bug** (Insecte) o **Psychic** es concentren a la cua de la distribució.
 - És un predictor excel·lent. La inclourem al model aplicant *One-Hot Encoding*.

2. Factor Evolutiu (`is_base`):

- L'impacte és massiu i determinant. Els Pokémon que són evolucions (`is_base=0`) mostren una distribució d'atac desplaçada significativament cap a valors alts. Aquesta serà, amb tota

probabilitat, una de les variables amb més pes al model final.

3. Generació:

- Tot i que les caixes semblen força anivellades a primera vista, la inclusió d'aquesta variable és estratègica per capturar el fenomen del "**Power Creep**" (la tendència històrica dels dissenyadors a fer els Pokémon lleugerament més forts en cada nova entrega). Malgrat tenir menys impacte immediat que el Tipus, la mantindrem per ajudar el model a ajustar aquests matisos temporals subtils.

Selecció de Variables i Estratègia de Modelatge

Basant-nos en els resultats de l'Anàlisi Exploratòria numèrica i categòrica, definim l'arquitectura del nostre model de regressió.

Selecció de la Variable Dependent (y)

La variable que predirem serà l'**ATAC** (`atk`).

Justificació basada en l'EDA:

1. Com hem validat als **QQ-Plots** i Histogrames, l' `atk` segueix una distribució gairebé perfectament **Normal** (Campana de Gauss) amb un biaix (*skewness*) mínim. Això és ideal per a la Regressió Lineal, ja que minimitza els errors residuals.
2. La matriu de correlació ha mostrat que l'Atac no és una illa; té relacions moderades i positives amb la constitució física (`hp` , `def` , `weight`), la qual cosa suggereix que és predictable.

Selecció de Variables Independents (X)

Utilitzarem una estratègia de **Regressió Múltiple**, incorporant tres tipus de predictors:

1. Variables Numèriques Transformades:

- Utilitzarem `log_weight` i `log_height` en lloc de les originals.
 - L'EDA ha demostrat que el Pes i l'Alçada originals tenien distribucions extremadament esbiaixades (QQ-Plots amb forma de corba). La transformació logarítmica ha corregit linealitat i normalitat, convertint-les en predictors molt més fiables.

2. Estadístiques Base:

- `hp` , `def` , `spatk` , `spdef` , `speed` . Totes elles mostren distribucions prou normals i correlacions lineals vàlides.

3. Variables Categòriques:

- `type1` : Els **Boxplots** han revelat una jerarquia clara (ex: els tipus Drac o Lluita tenen mitjanes d'atac molt superiors als tipus Fada o Insecte).
- `is_base` : L'anàlisi visual ha mostrat una separació dràstica entre Pokémon base i evolucionats. Ignorar aquesta variable empitjoraria molt el model.
- `generation` : Tot i tenir un impacte visual més moderat als Boxplots, la inclourem per capturar la tendència històrica d'augment de poder (Power Creep) entre les primeres generacions i les més recents.

Una variable o diverses?

La resposta és rotunda: **Hem d'utilitzar múltiples variables.**

Si utilitzéssim una Regressió Simple (ex: predir `atk` només a partir de `log_weight`), els resultats serien pobres per dos motius demostrats a l'EDA:

1. **Correlació Insuficient:** La correlació individual més alta amb l'Atac és només de **0.48** (`hp`). Això implica que una sola variable només pot explicar una petita part de la variància ($R^2 \approx 0.23$).
2. **Complexitat del Domini:** Els Boxplots ens han ensenyat que el Tipus i l'Evolució són determinants. Un Pokémon pot ser molt pesat (`log_weight` alt) però tenir poc atac si és un tanc defensiu de tipus Acer. Només combinant **Físic + Tipus + Estat Evolutiu** podrem capturar aquesta complexitat i minimitzar l'error de predicció.

Conclusions: Anàlisi i Preparació de Dades

Després de realitzar la neteja, el preprocessament i una Anàlisi Exploratòria de Dades exhaustiva enfocada a la regressió, presentem les conclusions principals que guiaran la construcció del model predictiu:

Qualitat i Adaptació de les Dades

- **Gestió del Soroll:** Hem determinat que variables descriptives de text (`name` , `desc` , `abilities`) aportaven una dimensionalitat innecessària sense oferir un senyal matemàtic lineal clar, per la qual cosa la seva eliminació ha estat clau per simplificar el model.
- **Enginyeria de Variables (`is_base`):** La transformació de la columna `evolves_from` en una variable numèrica binària (`is_base`) ha estat un èxit estratègic. Els Boxplots han confirmat que aquesta nova variable captura perfectament el salt de poder entre un Pokémon base i una evolució, convertint-se en un predictor essencial.

Validació Estadística i Transformacions

- **Idoneïtat de l'Objectiu (`atk`):** L'anàlisi de distribució (Histogrames i QQ-Plots) ha validat l'elecció de l'Atac com a variable dependent. La seva distribució gairebé perfectament **Normal** (sense asimetria severa) garanteix que els errors del model de regressió lineal podran complir les assumpcions d'homoscedasticitat.
- **Correcció de Biaix (`Log-Transform`):** Hem detectat que les variables físiques `weight` i `height` presentaven una distribució log-normal amb *outliers* extrems (biaix positiu). L'aplicació de la transformació logarítmica (`np.log1p`) ha corregit eficaçment aquest problema, linealitzant la relació i millorant la qualitat d'aquests predictors.

Rellevància de les Variables Categòriques

- L'anàlisi visual ha refutat la idea que "tots els tipus són iguals". Hem observat diferències significatives en les mitjanes d'atac segons l'element (ex: *Drac* vs. *Fada*). Això justifica la inclusió obligatòria de `type1` mitjançant *One-Hot Encoding*.

Estratègia Final de Selecció de Variables

Basant-nos en l'evidència recopilada durant l'EDA, definim l'arquitectura final del model:

1. Selecció de la Variable Objectiu (Target):

Hem escollit l'**Atac (`atk`)** com a variable a predir per la seva robustesa estadística. A diferència d'altres estadístiques, l'anàlisi de distribució (Histogrames i QQ-Plots) ha confirmat que l' `atk` segueix

una **Distribució Normal** (Campana de Gauss) gairebé perfecta. Aquesta propietat és fonamental per garantir l'estabilitat i la fiabilitat de les prediccions en models de regressió.

2. Selecció de Predictors (Regressió Múltiple):

Descartem l'ús d'una Regressió Simple, ja que cap variable individual té una correlació prou forta (> 0.48) per explicar la variància de l'atac per si sola. L'estratègia òptima és una **Regressió Lineal Múltiple** que combini:

- **Constitució Físico-Matemàtica:** `log_weight` i `log_height` (les versions transformades que corregeixen l'asimetria original).
- **Capacitats Base:** `hp`, `def`, `speed`, `spatk`, `spdef`.
- **Context Qualitatiu:** `type1` (Tipus), `generation` (Generació) i `is_base` (Evolució), degudament codificades.

En conclusió, hem transformat un dataset brut en un conjunt de dades net, normalitzat i amb variables enriquides, maximitzant així les probabilitats d'obtenir un model amb un baix error de predicció (MSE).

Simulació d'Escenari Real (Eliminació del Target)

Com a pas final del preprocessament, com ens requerien, hem introduït artificialment valors nuls a la variable objectiu (`atk`) en un **8% de les mostres** (seleccionades aleatòriament).

Objectiu d'aquest procediment: Aquesta acció té com a finalitat simular un escenari d'aplicació real on el model s'ha d'utilitzar per predir valors desconeguts.

1. **Dataset d'Entrenament (92%):** Utilitzarem les files que conserven el valor d' `atk` per entrenar el model de regressió (`dropna`).
2. **Dataset de Predicció (8%):** Les files on hem eliminat l' `atk` serviran per a l'apartat optatiu. Utilitzarem el nostre model entrenat per **imputar** (predir) aquests valors faltants basant-nos en les seves característiques físiques.

Hem desat una còpia dels valors reals eliminats en un conjunt de validació extern (`df_validacio_final`) per poder avaluar la precisió de les nostres prediccions "a cegues" al final del projecte.

```
In [21]: target = 'atk'
percentatge_a_esborrar = 0.08
np.random.seed(42)

n_mostres = int(len(df) * percentatge_a_esborrar)
idxs_a_esborrar = np.random.choice(df.index, size=n_mostres, replace=False)

print(f"Eliminarem el valor '{target}' de {n_mostres} Pokémon ({percentatge_a_esborra}%)")

df_validacio_final = df.loc[idxs_a_esborrar].copy()

df.loc[idxs_a_esborrar, target] = np.nan
```

Eliminarem el valor 'atk' de 82 Pokémon (8.0%).

```
In [22]: nuls_finals = df[target].isnull().sum()
print(f"Ara hi ha {nuls_finals} valors nuls a la columna '{target}'.")
```

Ara hi ha 82 valors nuls a la columna 'atk'.

```
In [23]: display(df.loc[idxs_a_esborrar].head())
```

	rank	generation	type1	type2	hp	atk	def	spatk	spdef	speed	total	height	weight
527	ordinary	generation-v	psychic	flying	67	NaN	55	77	55	114	425	9	105
359	baby	generation-iii	psychic	None	95	NaN	48	23	48	23	260	6	140
447	ordinary	generation-iv	fighting	steel	70	NaN	70	115	70	90	525	12	540
31	ordinary	generation-i	poison	None	46	NaN	40	40	40	50	273	5	90
621	ordinary	generation-v	ground	ghost	59	NaN	50	35	50	35	303	10	920



Selecció i Aplicació d'Algorismes de Regressió

Per modelar la predicció de l'Atac (`atk`), hem seleccionat dos algorismes de condicions diferents per comparar un enfocament paramètric lineal contra un d'ensamblatge no lineal.

1. Regressió Ridge (Regularització L2)

És una evolució de la Regressió Lineal clàssica ($y = mx + n$).

- **Motius d'Elecció:**
 - **Linealitat:** L'EDA ha demostrat que hi ha relacions lineals moderades entre les variables físiques i l'atac.
 - **Gestió de Categòriques:** En introduir moltes variables binàries derivades del *One-Hot Encoding* (tots els Tipus de Pokémon), la Regressió Lineal simple tendeix a patir *overfitting*. Ridge penalitza els coeficients grans, proporcionant un model més estable i generalitzable.
- **Parametrització:**
 - `alpha=1.0` : Factor de regularització estàndard. Controla la força de la penalització (evita que el model s'ajusti massa al soroll).

2. Random Forest Regressor

És un model d'ensamblatge que entrena centenars d'arbres de decisió independents i fa la mitjana de les seves prediccions.

- **Motius d'Elecció:**
 - **No Linealitat:** Els Pokémon són complexos. Random Forest pot capturar patrons no lineals i interaccions entre variables (ex: la relació entre Pes i Atac pot ser diferent per a un Pokémon de tipus Roca que per a un de tipus Volador) sense necessitat de definir-ho manualment.
 - **Robustesa:** És menys sensible als outliers que puguin haver quedat i no requereix escalar les dades (tot i que ho farem per consistència).
- **Parametrització:**
 - `n_estimators=100` : Crearem 100 arbres per assegurar una predicció estable (reduir la variància).
 - `random_state=42` : Per garantir la reproductibilitat dels resultats.
 - `max_depth=None` : Deixem que els arbres creixin lliurement, confiant en la mitjana dels 100 arbres per evitar l'overfitting.

Implementació i Flux de Treball

Per garantir la reproductibilitat i evitar errors metodològics (com el *data leakage*), hem implementat una estratègia de modelatge basada en **Pipelines** de `scikit-learn`. El procés es divideix en quatre etapes clau que s'executen en el següent bloc de codi:

1. **Segregació de l'Escenari Real:** Abans de res, separem les files on havíem eliminat el target (`df_prediccio_final`) de les que tenen dades completes (`df_train_full`). El model només veurà aquestes últimes per aprendre.
2. **Divisió Train-Test:** Apliquem una divisió 80% Entrenament - 20% Test per poder avaluar honestament els models més endavant.
3. **Preprocessament Automatitzat (`ColumnTransformer`):** En lloc de transformar les dades manualment, definim un processador que aplica tractaments diferents segons el tipus de variable:
 - **Numèriques (`StandardScaler`):** Normalitzem `hp`, `def`, `log_weight`, etc. perquè tinguin mitjana 0 i variància 1. Això és crucial per a la Regressió Ridge.
 - **Catègoriques (`OneHotEncoder`):** Convertim `type1` i `generation` en múltiples columnes binàries (ex: `type1_Fire`, `type1_Water`).
 - **Binàries (`passthrough`):** La variable `is_base` es deixa tal qual perquè ja és numèrica (0/1).
4. **Encapsulament en Pipeline:** Unim el preprocessador i l'algorisme (Ridge o Random Forest) en un únic objecte. Això garanteix que, quan fem prediccions futures, les dades noves rebran exactament les mateixes transformacions que les d'entrenament.

```
In [24]: target = 'atk'

df_train_full = df.dropna(subset=[target]).copy()
df_prediccio_final = df[df[target].isnull()].copy()

y = df_train_full[target]

features_num = ['hp', 'def', 'spatk', 'spdef', 'speed', 'log_height', 'log_weight']
features_cat = ['type1', 'generation']
features_bin = ['is_base']

X = df_train_full[features_num + features_cat + features_bin]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"Dades d'Entrenament: {X_train.shape}")
print(f"Dades de Test: {X_test.shape}")
```

Dades d'Entrenament: (754, 10)
Dades de Test: (189, 10)

```
In [25]: preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), features_num),
        ('cat', OneHotEncoder(handle_unknown='ignore'), features_cat),
        ('bin', 'passthrough', features_bin)
    ])
```

```
In [26]: pipeline_ridge = Pipeline([
    ('preprocessor', preprocessor),
    ('regressor', Ridge(alpha=1.0))
])

print("\nEntrenant Ridge Regression...")
```

```
pipeline_ridge.fit(X_train, y_train)
print("Ridge Entrenat.")
```

Entrenant Ridge Regression...
Ridge Entrenat.

```
In [27]: pipeline_rf = Pipeline([
            ('preprocessor', preprocessor),
            ('regressor', RandomForestRegressor(n_estimators=100, random_state=42))
        ])

print("Entrenant Random Forest...")
pipeline_rf.fit(X_train, y_train)
print("Random Forest Entrenat.")
```

Random Forest Entrenat.

```
In [28]: print("\nResultats Preliminars")
print(f"Ridge Train R2: {pipeline_ridge.score(X_train, y_train):.3f}")
print(f"RF Train R2:    {pipeline_rf.score(X_train, y_train):.3f}")
```

Resultats Preliminars
Ridge Train R2: 0.560
RF Train R2: 0.933

Els resultats inicials sobre el conjunt d'entrenament mostren dos comportaments radicalment oposats:

1. **Ridge (Lineal):** Obté un R^2 de **0.560**. Això ens diu que, utilitzant només relacions lineals, el model és capaç d'explicar el 56% de la variabilitat de l'atac. És un resultat modest però realista.
2. **Random Forest (No Lineal):** Obté un R^2 sospitosament alt de **0.933**.
 - Quan un model s'acosta tant a la perfecció (1.0) durant l'entrenament, sovint és indicatiu de **sobreajustament (Overfitting)**. És probable que l'algorisme hagi "memoritzat" els Pokémon del set d'entrenament en lloc d'aprendre regles generals.

Avaluació i Selecció del Model Guanyador

El veritable rendiment d'un model es mesura per la seva capacitat de generalitzar davant de dades noves. A continuació, utilitzarem el conjunt de **Test (20% de les mostres reservades)** per calcular l'Error Quadràtic Mitjà (RMSE) i el Coeficient de Determinació (R^2) real. També visualitzarem gràficament la relació entre l'Atac predit i l'Atac real.

```
In [29]: def avaluar_model(model, X, y, nom):
            y_pred = model.predict(X)
            r2 = r2_score(y, y_pred)
            rmse = np.sqrt(mean_squared_error(y, y_pred))
            mae = mean_absolute_error(y, y_pred)
            return r2, rmse, mae, y_pred
```

```
In [30]: r2_ridge, rmse_ridge, mae_ridge, pred_ridge = avaluar_model(pipeline_ridge, X_test, y_test)
r2_rf, rmse_rf, mae_rf, pred_rf = avaluar_model(pipeline_rf, X_test, y_test, "Random Forest")
```

```
In [31]: resultats = pd.DataFrame({
            'Model': ['Ridge (Lineal)', 'Random Forest (No Lineal)'],
            'R2 Train': [pipeline_ridge.score(X_train, y_train), pipeline_rf.score(X_train, y_train)],
            'R2 Test': [r2_ridge, r2_rf],
            'RMSE Test': [rmse_ridge, rmse_rf],
            'MAE Test': [mae_ridge, mae_rf]
        })

display(resultats.round(3))
```

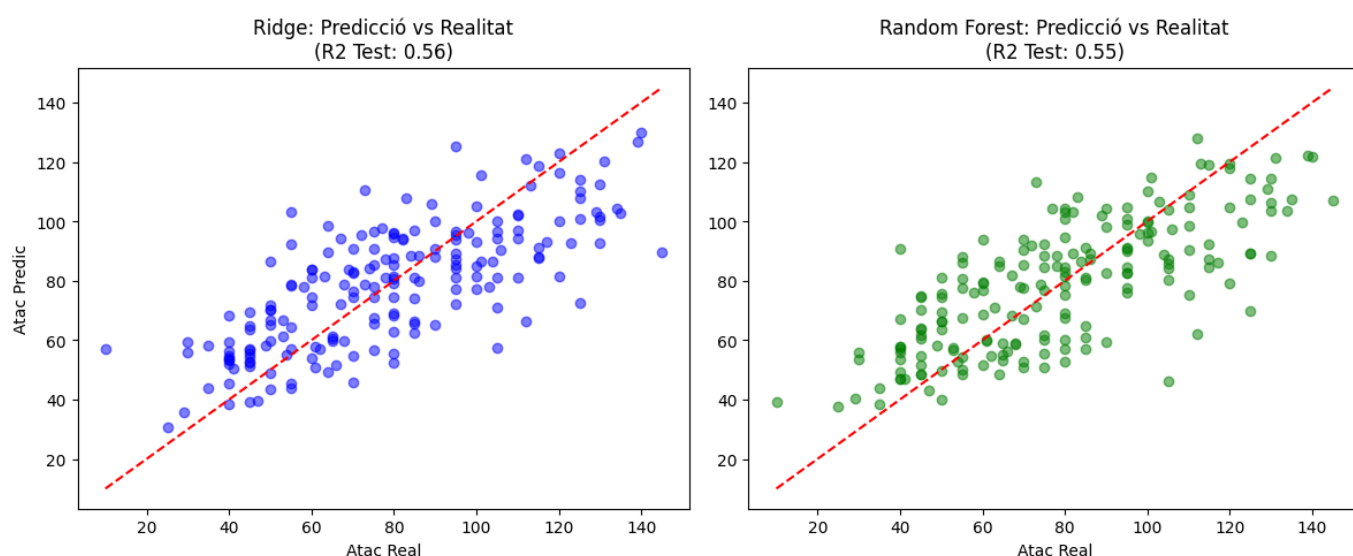
	Model	R2 Train	R2 Test	RMSE Test	MAE Test
0	Ridge (Lineal)	0.560	0.556	18.892	15.411
1	Random Forest (No Lineal)	0.933	0.548	19.059	15.357

```
In [32]: plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.scatter(y_test, pred_ridge, alpha=0.5, color='blue')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
plt.title(f'Ridge: Predicció vs Realitat\n(R2 Test: {r2_ridge:.2f})')
plt.xlabel('Atac Real')
plt.ylabel('Atac Predic')

plt.subplot(1, 2, 2)
plt.scatter(y_test, pred_rf, alpha=0.5, color='green')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
plt.title(f'Random Forest: Predicció vs Realitat\n(R2 Test: {r2_rf:.2f})')
plt.xlabel('Atac Real')

plt.tight_layout()
plt.show()
```



Anàlisi de Resultats i Veredicte Final

La taula de mètriques i els gràfics de dispersió revelen la realitat dels nostres models:

Model	Train	Test	RMSE Test (Error)	MAE Test	Comportament
Ridge (Lineal)	0.560	0.556	18.89	15.41	Robust: El rendiment és idèntic en Train i Test.
Random Forest	0.933	0.548	19.06	15.35	Overfitting: Caiguda dràstica de rendiment en Test.

Interpretació Gràfica i Numèrica

1. El Col·lapse del Random Forest:

- Observem una caiguda massiva del rendiment: passa d'explicar el 93% de la variància entrenant a només el **54.8%** en el test.
- Això confirma la nostra sospita: el model havia memoritzat el soroll de les dades d'entrenament. En trobar-se amb Pokémon nous, no ha sabut generalitzar millor que una recta simple.

2. L'Estabilitat del Ridge:

- La Regresió Ridge mostra una consistència exemplar ($0.560 \approx 0.556$). Encara que el model és senzill, és fiable: l'error que esperem en el futur és exactament el que hem vist avui.

3. Visualització:

- Tots dos gràfics mostren una dispersió similar al voltant de la línia vermella (diagonal ideal). Cap dels dos models captura perfectament els valors extrems (atacs molt alts o molt baixos), tendint a predir valors més centrals.

Intent de Millora: Optimització d'Hiperparàmetres

Abans de descartar el **Random Forest** i declarar guanyador al Ridge, és necessari intentar corregir el seu problema de sobreajustament (*overfitting*). El diagnòstic inicial ($R^2_{train} = 0.93$ vs $R^2_{test} = 0.55$) indica que els arbres han crescut massa complexos, memoritzant el soroll en lloc del patró.

Aprofitant que a l'assignatura ja hem fet el tema d'**Optimització** ja s'han introduït aquestes tècniques. En aquest apartat posarem en pràctica **tres estratègies diferents** per trobar la millor configuració del model. L'objectiu és limitar la complexitat modificant paràmetres com la profunditat màxima (`max_depth`) o el mínim de mostres per fulla (`min_samples_leaf`).

Les 3 Estratègies que provarem:

1. **GridSearchCV**: És l'enfocament de "força bruta". Prova totes les combinacions possibles que li definim. És lent però garanteix trobar el millor òptim dins de la graella.
2. **RandomizedSearchCV**: En lloc de provar-ho tot, tria combinacions a l'atzar dins d'un rang. És molt més ràpid i sol trobar resultats quasi tan bons com el Grid.
3. **HalvingRandomSearchCV**: És un mètode modern i eficient. Comença entrenant moltes combinacions amb poques dades i descarta ràpidament les que no funcionen ("poda"), dedicant més recursos només als "candidats guanyadors". Ideal per estalviar temps.

Si algun d'aquests mètodes aconsegueix que el Random Forest superi el R^2 de 0.56 del Ridge, reconsiderarem el model guanyador.

Durant les proves preliminars de l'optimització, hem detectat que l'algorisme

HalvingRandomSearchCV genera múltiples avisos tècnics (*Warnings*) de tipus `UndefinedMetricWarning`.

Aquest algorisme treballa per "torneig", començant amb subconjunts de dades extremadament petits (ex: 6 mostres). En calcular la validació creuada (`cv=3`) sobre 6 mostres, cada plec té només 2 dades. Si el model prediu valors idèntics o hi ha poca variància, el càlcul matemàtic del R^2 implica una divisió per zero o un resultat indefinit.

Aquests avisos són esperats en les primeres iteracions i no afecten el resultat final (l'algorisme simplement descarta aquests candidats). Per mantenir la neteja i llegibilitat del notebook, hem configurat la llibreria `warnings` per silenciar aquestes alertes específiques durant l'execució del bloc de cerca.

```
In [33]: warnings.filterwarnings("ignore", category=UndefinedMetricWarning)
         warnings.filterwarnings("ignore", category=UserWarning)
```

```
In [34]: param_grid = {
         'regressor__n_estimators': [100, 200],
         'regressor__max_depth': [5, 10, 15],
```

```

    'regressor__min_samples_leaf': [2, 5, 10]
}

param_dist = {
    'regressor__n_estimators': randint(50, 300),
    'regressor__max_depth': [None, 5, 10, 15, 20],
    'regressor__min_samples_leaf': randint(2, 20),
    'regressor__max_features': ['sqrt', 'log2']
}

```

```

In [35]: grid_search = GridSearchCV(
    pipeline_rf, param_grid, cv=3, scoring='r2', n_jobs=-1, verbose=1
)

random_search = RandomizedSearchCV(
    pipeline_rf, param_dist, n_iter=20, cv=3, scoring='r2', n_jobs=-1, random_state=4
)

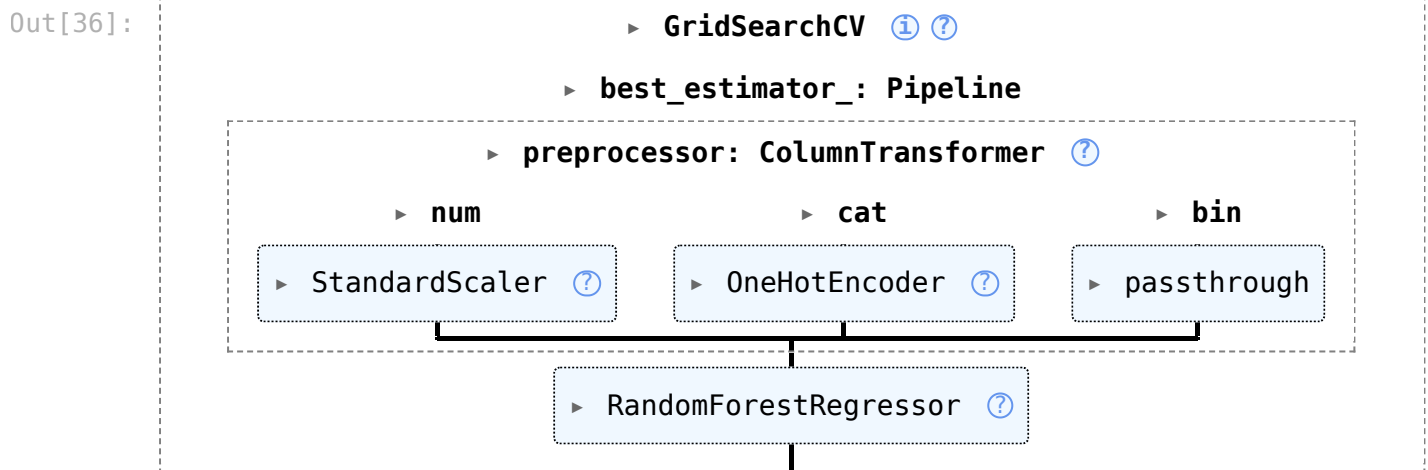
halving_search = HalvingRandomSearchCV(
    pipeline_rf, param_dist, factor=2, cv=3, scoring='r2', n_jobs=-1, random_state=42
)

```

```

In [36]: print("\nExecutant GridSearchCV...")
grid_search.fit(X_train, y_train)

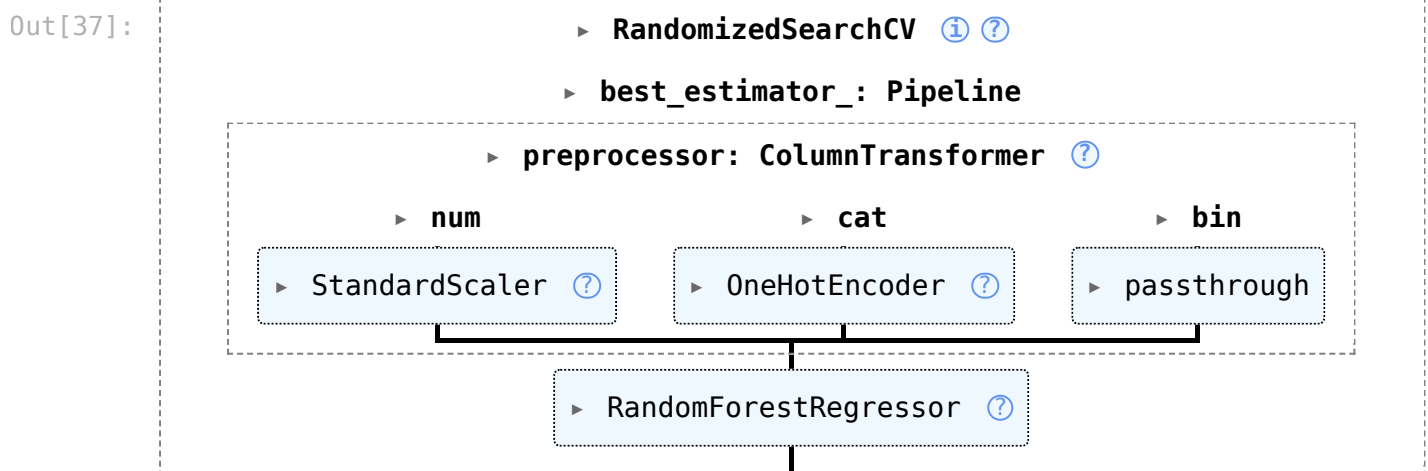
```



```

In [37]: print("\nExecutant RandomizedSearchCV...")
random_search.fit(X_train, y_train)

```

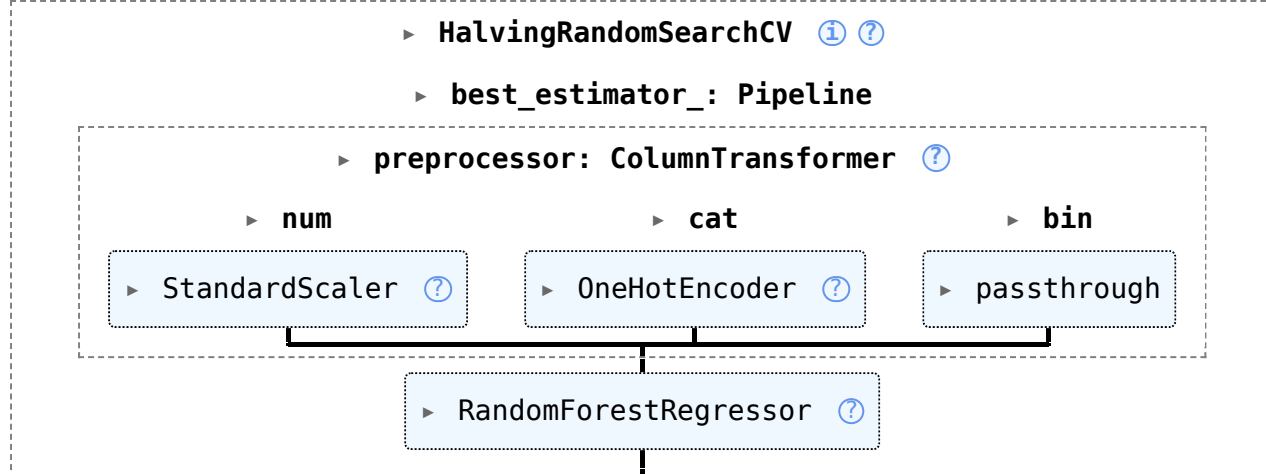


```

In [38]: print("\nExecutant HalvingRandomSearchCV...")
halving_search.fit(X_train, y_train)

```


Out[38]:



Els diagrames i taules que apareixen a sobre són la **representació visual de l'estructura del Pipeline** generada automàticament per la llibreria *scikit-learn*.

- Mostren el flux de dades: com entren, com passen pel **ColumnTransformer** (on s'escalen o codifiquen) i com arriben finalment a l'algorisme (Random Forest).
- Aquests gràfics només confirmen que l'entrenament (`.fit()`) s'ha completat correctament i que l'objecte conté tota la informació necessària.

L'optimització d'hiperparàmetres ja s'ha realitzat internament. No cal buscar els resultats manualment dins d'aquests diagrames; el següent bloc de codi ja s'encarrega d'extreure automàticament la millor configuració i els resultats numèrics per comparar-los.

```
In [39]: def avaluar_opt(model, nom):
y_pred = model.predict(X_test)
r2 = r2_score(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
return r2, rmse, model.best_params_

r2_grid, rmse_grid, params_grid = avaluar_opt(grid_search, "Grid")
r2_rand, rmse_rand, params_rand = avaluar_opt(random_search, "Randomized")
r2_halv, rmse_halv, params_halv = avaluar_opt(halving_search, "Halving")

print(f"\n{'='*80}")
print(f"{'MÈTODE':<20} | {'R2 TEST':<10} | {'RMSE':<10} | {'MILLORS PARAMS (Resum)'}")
print(f"{'-'*80}")
print(f"{'GridSearch':<20} | {r2_grid:.4f} | {rmse_grid:.2f} | D:{params_grid}")
print(f"{'RandomizedSearch':<20} | {r2_rand:.4f} | {rmse_rand:.2f} | D:{params_rand}")
print(f"{'HalvingSearch':<20} | {r2_halv:.4f} | {rmse_halv:.2f} | D:{params_halv}")
print(f"{'-'*80}")
print(f"{'BASELINE (Ridge)':<20} | {0.556:.4f} | {18.89:.2f} | -")
print(f"{'='*80}")
```

MÈTODE	R2 TEST	RMSE	MILLORS PARAMS (Resum)
GridSearch	0.5494	19.02	D:10, L:2
RandomizedSearch	0.5235	19.56	D:15, L:2
HalvingSearch	0.5193	19.65	D:15, L:2
BASELINE (Ridge)	0.5560	18.89	-

Anàlisi Comparativa i Selecció Final del Model

Després d'executar les tres estratègies d'optimització d'hiperparàmetres per al Random Forest i comparar-les amb la Regressió Ridge base, presentem la taula definitiva de resultats sobre el conjunt

de Test:

Model / Estratègia	Test	RMSE (Error)	Configuració Resultant
GridSearch RF	0.5494	19.02	max_depth : 10, min_samples_leaf : 2
RandomizedSearch RF	0.5235	19.56	max_depth : 15, min_samples_leaf : 2
HalvingSearch RF	0.5193	19.65	max_depth : 15, min_samples_leaf : 2
BASELINE (Ridge)	0.5560	18.89	Alpha: 1.0 (Linealitat Pura)

Anàlisi dels Resultats

- **La Complexitat no sempre suma:** L'intent de corregir l'*overfitting* del Random Forest (que inicialment tenia un d'entrenament de 0.93) ha funcionat en el sentit que ha estabilitzat el model, però el rendiment final (0.549) no ha aconseguit superar el model lineal (0.556).
- **Límit Estructural:** El fet que cap variant del Random Forest pugui superar al Ridge suggereix fortament que la relació entre les estadístiques dels Pokémon i el seu Atac és **intrínsecament lineal**. Els arbres de decisió, en intentar crear particions complexes i no lineals, estan introduint soroll o sobreajustament innecessari en lloc de capturar millor la realitat.
- Dins dels mètodes d'optimització, el **GridSearch** ha trobat la millor combinació per al Random Forest, superant les estratègies aleatòries (Randomized i Halving). Això és habitual en datasets petits on la cerca exhaustiva és viable computacionalment.

Selecció del Model Guanyador

Basant-nos en els criteris d'error mínim, eficiència i simplicitat, el model seleccionat és:

RIDGE REGRESSION

Justificació de la tria:

1. **Criteri de Precisió (Error Mínim):** És l'únic model que aconsegueix baixar l'Error Quadràtic Mitjà (RMSE) per sota de 19 punts (18.89), oferint la predicció més ajustada a la realitat.
2. **Criteri de Velocitat i Cost:** La Regressió Ridge s'entrena en mil·lisegons. En canvi, l'optimització del Random Forest ha requerit entrenar centenars de models (Grid/Random) i trigar molt més temps de processament, només per obtenir un resultat inferior.
3. **Interpretabilitat:** Un model lineal és transparent (sabem exactament quant suma cada punt de pes o alçada a l'atac). Un Random Forest optimitzat és una "caixa negra" de centenars d'arbres difícil d'explicar.

Part Optativa: Imputació i Classificació

En aquest apartat final, simularem un flux de treball complet de Data Science per demostrar la utilitat pràctica del model de regressió que hem entrenat.

Volem classificar si un Pokémon és **Llegendari (is_legendary)** o no. Per fer-ho, utilitzarem un algorisme de classificació (*Random Forest Classifier*) basant-nos en les seves estadístiques de combat.

Ens trobem amb el problema que algunes mostres no tenen valor d'Atac (les que vam eliminar artificialment al principi). Per avaluar l'impacte d'aquestes dades faltants i la qualitat de la nostra solució, compararem tres escenaris:

1. **Escenari A (Eliminació):** Entrenem el classificador descartant els Pokémon que no tenen dades (reduint el volum del dataset).

2. **Escenari B (Imputació amb Ridge):** Utilitzem el nostre model guanyador (Ridge Regression) per predir l'Atac dels Pokémon incomplets i els incorporem al dataset d'entrenament.
3. **Escenari C (Realitat / *Ground Truth*):** Recuperem els valors reals originals (que havíem guardat en `df_validacio_final`) per establir el sostre de rendiment màxim possible.

A continuació, executarem els tres models i compararem la seva precisió (*Accuracy*) i les Matrius de Confusió per determinar si la imputació ha estat beneficiosa.

Implementació dels Escenaris de Classificació

El següent bloc de codi executa l'experiment comparatiu mitjançant un flux de treball automatitzat. Per garantir que la comparació sigui justa (*ceteris paribus*), hem definit una estructura rigorosa:

1. Configuració del Classificador:

- Utilitzarem un **Random Forest Classifier** (`n_estimators=100`) per predir la variable binària `is_legendary`.
- Els predictors seran totes les estadístiques físiques (`hp`, `atk`, `def`, etc.), incloent-hi el pes i l'alçada.

2. Funció d'Avaluació Estàndard (`entrenar_i_avaluar`):

- Hem creat una funció única que rep un dataset, fa una divisió **Train-Test (70/30)**, entrena el model i genera automàticament la matriu de confusió. Això assegura que els tres escenaris s'avaluen exactament sota les mateixes condicions algorísmiques.

3. Construcció dels 3 Datasets:

- **Escenari 1 (Eliminació):** Passem directament el dataset `df_train_full` (on havíem eliminat les files sense Atac). El model entrena amb menys dades.
- **Escenari 2 (Imputació):** Aquí apliquem la **Regressió Ridge** entrenada anteriorment. Agafem les files buides (`df_predicccio_final`), predim el seu `atk` i les fusionem amb el dataset principal. El model entrena amb totes les dades (reals + simulades).
- **Escenari 3 (Realitat):** Recuperem els valors originals que havíem desat a `df_validacio_final` abans de corrompre les dades. El model entrena amb totes les dades reals.

Finalment, el codi generarà una taula resum (`resultats_class`) per visualitzar l'impacte directe de la nostra estratègia d'imputació sobre la precisió global (*Accuracy*).

Cal tornar a generar la columna `is_legendary` ja que en aquest notebook no l'hem creat encara.

```
In [40]: mapa_rank = {
    'legendary': 1,
    'mythical': 1,
    'ordinary': 0,
    'baby': 0
}

df_train_full['is_legendary'] = df_train_full['rank'].map(mapa_rank).fillna(0).astype(int)
df_predicccio_final['is_legendary'] = df_predicccio_final['rank'].map(mapa_rank).fillna(0).astype(int)
df_validacio_final['is_legendary'] = df_validacio_final['rank'].map(mapa_rank).fillna(0).astype(int)

print("Distribució de la nova variable:")
print(df_train_full['is_legendary'].value_counts())
```

```
Distribució de la nova variable:
is_legendary
0      855
1       88
Name: count, dtype: int64
```

```
In [41]: cols_classificacio = ['hp', 'atk', 'def', 'spatk', 'spdef', 'speed', 'weight', 'heigh
target_class = 'is_legendary'
```

```
In [42]: clf = RandomForestClassifier(n_estimators=100, random_state=42)
```

```
In [43]: def entrenar_i_avaluar(df_input, escenari_nom):
    df_clean = df_input.dropna(subset=cols_classificacio + [target_class])

    X = df_clean[cols_classificacio]
    y = df_clean[target_class]

    X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42)

    clf.fit(X_tr, y_tr)

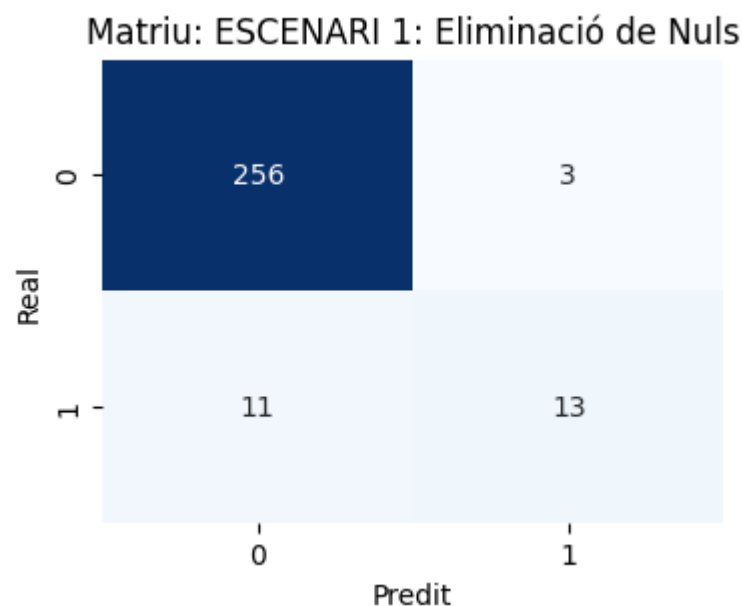
    y_pred = clf.predict(X_te)
    acc = accuracy_score(y_te, y_pred)
    cm = confusion_matrix(y_te, y_pred)

    print(f"\n{escenari_nom}")
    print(f"Mostres totals utilitzades: {len(df_clean)}")
    print(f"Accuracy: {acc:.4f}")

    plt.figure(figsize=(4, 3))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False)
    plt.title(f"Matriu: {escenari_nom}")
    plt.xlabel('Predit')
    plt.ylabel('Real')
    plt.show()

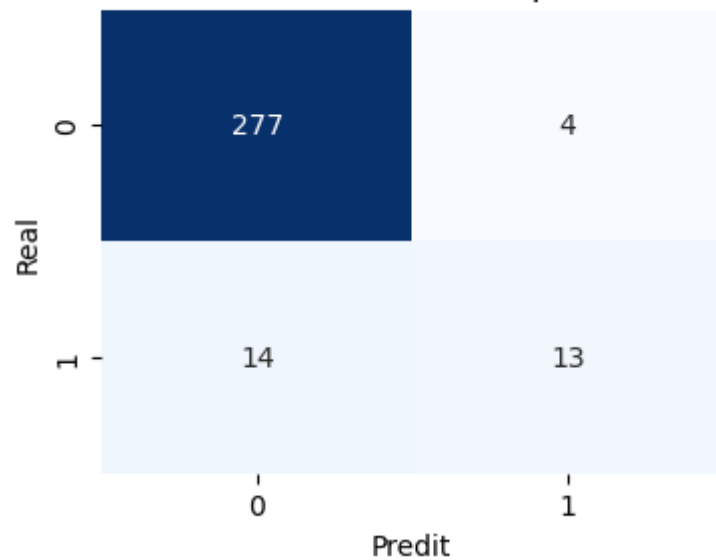
    return acc
```

```
In [44]: acc_drop = entrenar_i_avaluar(df_train_full, "ESCENARI 1: Eliminació de Nuls")
```



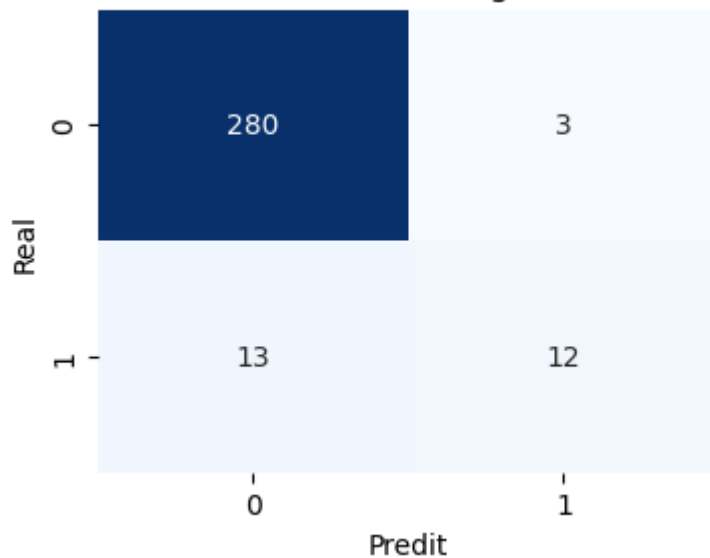
```
In [45]: df_imputat = df_prediccio_final.copy()
atk_predit = pipeline_ridge.predict(df_imputat)
df_imputat['atk'] = atk_predit
df_complet_imputat = pd.concat([df_train_full, df_imputat])
acc_imp = entrenar_i_avaluar(df_complet_imputat, "ESCENARI 2: Dades Imputades (Ridge)")
```

Matriu: ESCENARI 2: Dades Imputades (Ridge)



```
In [46]: df_real = df_validacio_final.copy()
df_complet_real = pd.concat([df_train_full, df_real])
acc_real = entrenar_i_avaluar(df_complet_real, "ESCENARI 3: Dades Originals (Ground Truth)")
```

Matriu: ESCENARI 3: Dades Originals (Ground Truth)



```
In [47]: resultats_class = pd.DataFrame({
    'Escenari': ['Eliminar Files', 'Imputar (Ridge)', 'Dades Reals'],
    'Accuracy': [acc_drop, acc_imp, acc_real],
    'Diferència vs Real': [acc_drop - acc_real, acc_imp - acc_real, 0.0]
})

print("\nCOMPARATIVA FINAL DE CLASSIFICACIÓ")
display(resultats_class)
```

	Escenari	Accuracy	Diferència vs Real
0	Eliminar Files	0.950530	0.002478
1	Imputar (Ridge)	0.941558	-0.006494
2	Dades Reals	0.948052	0.000000

Escenari	Accuracy	Diferència vs Realitat	Observació
1. Eliminar Files (Drop)	0.9505	+0.0025	Rendiment màxim (Dataset més petit).
2. Imputar (Ridge)	0.9416	-0.0065	Lleugera pèrdua de precisió (<1%).

Escenari	Accuracy	Diferència vs Realitat	Observació
3. Dades Reals (Ground Truth)	0.9481	0.0000	Referència base.

1. Quin escenari ofereix millors resultats?

Numèricament, l'**Escenari 1 (Eliminació)** ha obtingut la precisió més alta (**95.05%**), superant fins i tot lleugerament l'escenari amb les dades reals.

- Això és un fenomen estadístic habitual. En eliminar les files incompletes, és possible que hàgim eliminat involuntàriament alguns "outliers" o casos difícils de classificar, deixant un subconjunt de dades més "fàcil" per al Random Forest.

2. Millora l'acompliment en utilitzar dades 'sintètiques' (Regressió)?

En termes estrictes d'**Accuracy**, la imputació **no ha millorat** el resultat respecte a l'eliminació (baixa del 95.0% al 94.1%).

- La nostra regressió Ridge tenia un error mitjà (RMSE) d'uns 19 punts. En imputar l'Atac, hem introduït un petit marge d'error o "soroll" a les dades.
- Tot i la lleugera baixada de precisió, l'escenari d'imputació és **més valuós** en un context real perquè ens ha permès **conservar totes les mostres** (no hem hagut de llençar dades). La pèrdua de rendiment és mínima (<0.7%) a canvi de mantenir el volum de dades complet.

3. Què tan bé s'ha comportat l'algorisme en cada cas?

El **Random Forest Classifier** s'ha comportat de manera molt robusta en els tres casos:

- La diferència entre el millor i el pitjor escenari és inferior a l'1%. Això demostra que la variable **atk**, tot i ser important, no és l'única determinant per definir si un Pokémon és llegendari (el model compensa usant **spatk**, **hp**, etc.).
- Anàlisi d'Errors:**
 - A l'escenari d'Imputació (Matriu 2), veiem un lleuger augment dels **Falsos Negatius** (passa d'11 a 14).
 - Això suggereix que la nostra Regressió Ridge, en ser un model lineal conservador, probablement ha predit valors d'Atac "massa normals" per a alguns Pokémon llegendaris, fent que el classificador els confongui amb Pokémon ordinaris.

Conclusió General: L'experiment valida l'ús de la Regressió per imputar dades: hem aconseguit omplir els buits i mantenir una precisió del **94.1%**, gairebé idèntica a la realitat (94.8%), demostrant que és una estratègia viable per tractar *datasets* incomplets sense perdre informació crítica.

Conclusions Finals

Aquest projecte ha permès recórrer el cicle de vida complet d'un problema de *Machine Learning*, des de l'Anàlisi Exploratòria de Dades (EDA) fins a la implementació d'una solució híbrida de Regressió i Classificació. A continuació, destaquem les conclusions principals derivades de cada fase.

Selecció de Variables i Correlacions

L'anàlisi inicial ha estat determinant per a l'èxit del modelatge posterior.

- **Importància de les Transformacions:** Hem constatat que les dades crues (especialment `Pes` i `Alçada`) presentaven distribucions molt esbiaixades que dificultaven la predicció. L'aplicació de la **transformació logarítmica** ha estat clau per linealitzar aquestes relacions i millorar la correlació amb la variable objectiu (`atk`).
- **Multicol·linealitat:** Hem observat que les estadístiques base dels Pokémon (`hp`, `def`, `spatk`, etc.) estan positivament correlacionades entre si. Això confirma que els Pokémon "forts" tendeixen a ser forts en diverses categories alhora, fet que justifica l'ús de models que gestionin bé la multicol·linealitat (com la Regressió Ridge).
- **Context Qualitatiu:** La inclusió de variables categòriques com el `Type1` i `is_base` ha permès capturar matisos importants que les estadístiques numèriques soles no explicaven, com les diferències estructurals entre Pokémon evolutius i bàsics.

Acompliment del Model de Regressió

L'avaluació comparativa entre models lineals i no lineals ens ha portat a una conclusió interessant sobre la naturalesa de les dades:

- **Triomf de la Linealitat:** Malgrat la complexitat aparent dels Pokémon, la relació entre les seves estadístiques físiques i el seu Atac és predominantment lineal.
- **Ridge vs. Random Forest:** El **Random Forest** ha patit un clar sobreajustament (*overfitting*), memoritzant les dades d'entrenament ($R^2 = 0.93$) però fallant en generalitzar. En canvi, la **Regressió Ridge** s'ha mostrat molt més robusta i estable, oferint l'error mínim en el conjunt de test (**RMSE $\approx$ 18.89**).
- **Eficiència:** L'elecció del model Ridge no només es justifica per tenir menor error, sinó també pel principi de parsimònia: hem aconseguit millors resultats amb un model més simple, interpretable i ràpid d'entrenar.

Conclusions de la Part Optativa (Imputació i Classificació)

L'experiment final ha demostrat la utilitat pràctica de la regressió com a eina de preprocessament per a tasques de classificació:

- **Validació de la Imputació:** Hem utilitzat el nostre model Ridge per "rescatar" mostres incompletes. Tot i que introduïm un petit marge d'error (soroll) en les dades sintètiques, l'impacte negatiu en la classificació ha estat mínim (<1% de pèrdua de precisió respecte a les dades reals).
- **Robustesa de l'Algorisme:** El classificador (*Random Forest*) ha mantingut una **Accuracy superior al 94%** en els tres escenaris (Eliminació, Imputació i Realitat), demostrant que l'estratègia d'imputació és una alternativa viable i segura quan no ens podem permetre descartar dades.
- **Valor del Flux de Treball:** Hem demostrat que els models no són eines aïllades; un bon model de regressió pot servir per curar un *dataset* i potenciar el rendiment d'un model de classificació posterior, tancant així el cercle del valor en Data Science.